

VDR-LLM-Prolog on FPGA

Exact Integer Arithmetic in Custom Silicon: A 10-Core Q335 Processor on Zynq-7020

Registry: [\[HOWL-VDR-21-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-14-2026\]](#) → ... → [\[HOWL-VDR-21-2026\]](#)

DOI: 10.5281/zenodo.20251944

Date: May 2026

Domain: Computer Architecture / Hardware Acceleration

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

Abstract

VDR-LLM-Prolog is an architecture for language models built on exact integer arithmetic, where every value is a triple [Value, Denominator, Remainder] of integers, data lives in scoped knowledge bases at integer addresses, and the language model orchestrates 448 deterministic primitives rather than generating computation as text. The arithmetic foundation uses Q335 — a fixed denominator of 2^{335} providing 100 decimal digits of precision — where addition is one integer add and multiplication is one integer multiply followed by division by the fixed denominator. This paper presents a custom FPGA processor design that accelerates Q335 arithmetic and Prolog knowledge base operations on a Xilinx Zynq-7020 system-on-chip. The design exploits a structural property of the Q335 representation: because the denominator is a fixed power of two, division by the denominator is bit extraction — bits above position 335 are the quotient, bits below are the remainder. In digital logic, fixed-position bit extraction is wiring, not computation: zero logic cells, zero clock cycles beyond routing delay, zero power consumption. The processor implements ten VDR-Q335 cores, each with a 384-bit ALU (1-cycle addition, 9-cycle multiplication via iterative 128-bit tiling on DSP48E1 slices, 1-cycle comparison, 19-cycle cross-multiply for Prolog unification), a 2KB remainder tree in block RAM, and a 53-instruction ISA spanning wide arithmetic, remainder tree management, Prolog matching, and batch processing. A batch dispatcher distributes work across cores via DMA, and a binary reduction network combines partial results in 5 cycles. The system fits within 54.2% of available lookup tables and 73.4% of flip-flops, leaving headroom for timing closure and future additions. At 150 MHz, the design achieves single Prolog fact queries across 200 facts in approximately 200 nanoseconds, attention dot products for 64-dimensional vectors in approximately 6 microseconds, surrogate softmax over 100 logits in approximately 3.3 microseconds, and SGD parameter updates in approximately

15 cycles per element. The ARM host processor runs the Zig VDR-LLM-Prolog runtime — orchestrated inference, knowledge base management, sessions, grammar-directed generation, and language model passes — while the FPGA accelerates parallel data-plane operations through a register-mapped control interface and high-bandwidth DMA. Every accelerated operation produces results bit-identical to the software implementation, verified against the project’s 884-test validation suite with zero arithmetic errors. The architecture scales linearly: the same core design targets 60 cores on Zynq-7045, 120 on Zynq-7100, and 200+ on UltraScale+ devices.

1. The System in Brief

The VDR-LLM-Prolog system replaces floating-point arithmetic with exact integer triples. Every number in the system is three integers: Value, Denominator, and Remainder. The Value and Denominator are plain integers forming a rational fraction. The Remainder carries exact unresolved structure — it is not error and it is not rounding residual. When the Remainder is zero, the triple behaves as the rational number V/D . When it is nonzero, the triple carries exact information beyond what the denominator frame could absorb. The Remainder is the only slot that nests, forming a structural tree where each level is itself exact. This solves the denominator explosion problem that makes naive exact rational arithmetic impractical: growth goes to tree depth, not denominator width.

The system fixes the denominator at 2^{335} , a configuration called Q335. This provides a precision floor of approximately 100 decimal digits — 10^{66} times below the Planck length, exceeding float64 precision by 85 orders of magnitude. Twenty-two transcendental constants (π , e , $\ln 2$, $\zeta(3)$, $\sqrt{2}$, φ , and others) are stored as roughly 102-digit integer numerators over this shared denominator. Adding two constants is one integer addition of numerators. Multiplying two constants is one integer multiplication followed by a divmod at bit position 335 — bits above become the new Value, bits below become the Remainder. The denominator never changes. The arithmetic has been validated across 884 tests spanning 37 domains with zero computation errors.

The language model in VDR-LLM-Prolog does not compute. It orchestrates. It selects from 448 deterministic primitives — 404 pure mathematical functions and 44 grant-gated operational primitives — by emitting structured command tokens of roughly 8 tokens each. Data lives in knowledge bases organized as a scoped tree addressable by integer. The language model references data by typed paths that resolve to integer identifiers. Data never flows through the token stream. State persists in knowledge bases across sessions. A Prolog engine handles logical deduction. A grammar system provides structural tokens for free with guaranteed correctness. Safety is structural: knowledge base visibility checks and grant authorization are integer operations in the primitive layer, executed before the language model is involved. The result is 85-97% fewer language model tokens than conventional approaches, flat per-turn cost instead of quadratic attention growth, and exact reproducible results on any platform.

The arithmetic foundation is specified in [\[HOWL-VDR-1-2026\]](#). The Q335 constant

basis is specified in [HOWL-MATH-4-2026]. Physical computation across 14 domains is validated in [HOWL-VDR-13-2026]. The exact language model pipeline is specified in [HOWL-VDR-4-2026]. The complete system specification is [HOWL-VDR-14-2026]. GPU performance analysis is in [HOWL-VDR-18-2026]. This paper takes the system from software to silicon.

2. Why Hardware Acceleration

The performance profile of VDR-LLM-Prolog creates a specific acceleration opportunity. A single Q335 multiplication costs approximately 200 integer operations — roughly 200 times slower per operation than a float16 multiply. But the system generates 85-97% fewer language model tokens than conventional architectures, and every token not generated is a full forward pass not executed. At 95% token reduction, the net forward pass cost is approximately 10 times that of conventional float inference for a single turn. Over multiple turns the balance shifts further in VDR’s favor because per-turn cost is flat (state in knowledge bases at integer addresses) while conventional attention cost grows quadratically with conversation length.

Within this profile, specific operations dominate computation time and are natural candidates for hardware acceleration.

Prolog fact matching occurs on every knowledge base query. The system searches facts by predicate within the scope chain — the active topic’s knowledge base and each ancestor up to root. With tens of thousands of facts in a production deployment, sequential scanning becomes the bottleneck. Each fact match is an independent comparison — the predicate identifier is an integer, and matching is integer equality. This is embarrassingly parallel.

Q335 batch arithmetic occurs on every training step. Stochastic gradient descent subtracts the learning rate times the gradient from each parameter. Each parameter update is independent: one Q335 multiplication (learning rate times gradient) and one Q335 subtraction (parameter minus update). With millions of parameters, bulk updates dominate training time. Each update is the same operation on different data — perfectly uniform parallel work.

Attention score computation is the most compute-intensive operation in the transformer forward pass. Computing Q times K-transpose requires a dot product of exact Q335 fractions for every query-key position pair. Each row of the attention matrix is independent. Within each row, the dot product is a sequence of Q335 multiplies and adds — uniform, deterministic, parallelizable.

Constraint checking occurs at every normalization point. The system evaluates each constraint against the current state independently. With 15 or more constraints per lifecycle phase, parallel evaluation reduces latency from the sum of all constraint evaluation times to the time of the single most expensive constraint.

All of these operations share three properties: they are exact integer arithmetic, they are embarrassingly parallel or map-reduce, and they are the same operation repeated across

different data. This is the workload profile that custom hardware — fixed-function integer processing units operating in parallel — handles best.

The host processor handles everything else: the orchestrated inference loop, knowledge base tree management, session management, grammar parsing and generation, command token dispatch, language model forward and backward passes, functional remainder resolution, and all control flow decisions. The division is clean: the host runs the control plane, the FPGA runs the data plane. The IOSE declarations that specify every primitive's interface are the contract between them.

3. The Q335 Hardware Opportunity

The decision to fix the denominator at a power of two was made for mathematical reasons. A shared denominator means addition of two Q335 values is one integer addition of numerators — no lowest common denominator computation, no cross-multiplication. A power-of-two denominator means multiplication followed by divmod can be implemented as a right shift. Growth goes to remainder tree depth instead of denominator width, solving the explosion problem that makes naive exact arithmetic impractical.

This mathematical decision turns out to be an optimal hardware design decision. In digital logic, division by a power of two is not computation. It is routing. A binary number stored in a register is a sequence of flip-flops, each holding one bit. Dividing by 2^{335} means the bits above position 335 are the quotient and the bits below position 335 are the remainder. Extracting these two results requires connecting wires from the appropriate flip-flops to the appropriate output registers. There are no logic gates in the path. There is no multiplexer. There is no arithmetic logic unit involvement. The “operation” is performed by the physical layout of copper traces on the chip.

In concrete terms: after a Q335 multiplication produces a 768-bit result in a register pair, the SHR335 instruction extracts bits [767:335] as the quotient (the new Value) and bits [334:0] as the remainder. This takes one clock cycle — the minimum for any instruction due to pipeline staging — but consumes zero lookup tables, zero flip-flops beyond the destination registers, and effectively zero switching power because no logic transitions occur. The Q335 divmod that is the computational core of every VDR multiplication — the operation that makes the fixed-frame scheme work — is free in hardware.

The carry-select adder for 384-bit addition divides the operands into six 64-bit blocks. Each block simultaneously computes the sum assuming carry-in of zero and carry-in of one. As the actual carry propagates from the least significant block upward, a multiplexer at each stage selects the correct precomputed result. The entire 384-bit addition completes in one clock cycle using approximately 400 lookup tables and 50 flip-flops. This is the implementation of Q335 addition — the operation that combines two transcendental constants or adds two parameter updates.

The iterative multiplier handles 384-by-384-bit multiplication by tiling the operands into 128-bit segments. A 128-bit multiplier (implemented using 5 DSP48E1 hardware

multiply-accumulate slices) computes one partial product per cycle. The full multiplication requires a 3-by-3 grid of partial products — 9 cycles — with accumulation of partial products into the 768-bit result. Each cycle performs identical work on different segments of the operands. The computation is perfectly regular with no data-dependent branching.

The 384-bit comparator cascades six 64-bit comparisons with early termination. If the most significant 64-bit segments differ, the result is determined in one cycle without examining the remaining segments. In the worst case (all segments equal until the last), the full comparison takes one cycle because all segment comparisons run in parallel with the cascade logic selecting the first difference.

Cross-multiplication for Prolog unification of VDR fractions — testing whether a/b equals c/d by checking whether a times d equals c times b — requires two invocations of the 9-cycle multiplier followed by a 768-bit comparison. Total latency: 19 cycles. This is the hardware cost of determining whether two exact rational numbers are equal, which is the core operation in Prolog fact matching when arguments are VDR fractions.

The complete arithmetic profile at 150 MHz:

Operation	Cycles	Time	LUTs	Description
Q335 addition	1	6.7 ns	400	Carry-select, $6 \times$ 64-bit blocks
Q335 subtraction	1	6.7 ns	400	Same adder, complement input
Q335 multiplication	9	60 ns	200	Iterative 128-bit, 3×3 tiling
Q335 divmod	1	6.7 ns	0	Fixed wiring — bit extraction
Q335 comparison	1	6.7 ns	100	Cascaded 64-bit, early termination
Fraction unification	19	127 ns	80	Two multiplies plus 768-bit compare

Every operation is deterministic, produces the same result regardless of input values, and completes in a fixed number of cycles (comparison's early termination affects internal power consumption but not external timing). This is the workload profile that FPGA fabric handles at peak efficiency: fixed-width, fixed-latency, uniform integer operations with no branching.

4. VDR-Q335 Core Architecture

Each VDR-Q335 core is a custom processor optimized for 384-bit exact integer arithmetic with integrated support for remainder tree management and Prolog term matching.

Ten cores are instantiated on the target device. The core has a four-stage in-order pipeline — fetch, decode, execute, writeback — running at 150 MHz, yielding 150 million instructions per second per core for single-cycle operations.

4.1 Working Width

The working width is 384 bits. Q335 numerators require approximately 340 bits (102-digit integers have bit widths between 333 and 342 across the 22 basis constants). The 384-bit width provides a 44-bit margin for intermediate overflow during multiply-accumulate sequences. The width is also evenly divisible into six 64-bit segments, which simplifies the carry-select adder and comparator designs, and into three 128-bit segments, which maps naturally to the iterative multiplier's tiling strategy.

4.2 Register File

The register file provides the working storage for all core operations.

Eight Value registers (V0 through V7) hold 384-bit values — Q335 numerators, intermediate results, accumulators, and operands for arithmetic operations. Eight Remainder registers (R0 through R7) hold 384-bit values used for remainder components, comparison operands, and temporaries. Four Index registers (I0 through I3) hold 32-bit values used for knowledge base identifiers, fact indices, batch position counters, and loop control.

Four Batch control registers manage the batch processing interface: BF (16 bits, batch format and predicate identifier), BC (32 bits, batch count), BD (8 bits, batch depth), and BI (32 bits, batch index — current position within the batch).

Six single-bit flags record the results of comparison and arithmetic operations: EQ (equality), LT (less than), GT (greater than), OV (overflow from addition or multiplication), DONE (core has completed its assigned work), and CLOSED (the current VDR triple has zero remainder — the value is a closed rational).

The total register state per core is 6,264 bits — 783 bytes. This is implemented in distributed RAM using lookup tables rather than block RAM, keeping the register file close to the ALU in the FPGA fabric and enabling single-cycle read and write access.

A memory address register (MA, 32 bits) holds the current BRAM address for remainder tree and instruction memory access.

4.3 Arithmetic Logic Unit

The ALU performs all computation within the core. It contains five functional units that share the execute pipeline stage.

The 384-bit adder and subtractor uses a carry-select architecture. The 384-bit operands are divided into six 64-bit blocks. Each block contains two adders computing the sum for carry-in zero and carry-in one simultaneously. As the carry propagates from the least

significant block, a 2-to-1 multiplexer at each block boundary selects the correct precomputed result. Subtraction complements the second operand and sets carry-in to one. The entire operation completes in one cycle. Resource cost: approximately 400 lookup tables and 50 flip-flops.

The 128-bit iterative multiplier computes 384-by-384-bit products by tiling. The 384-bit operands are divided into three 128-bit segments each, producing a 3-by-3 grid of 9 partial products. Each partial product uses 5 DSP48E1 slices in a cascade configuration — the Zynq-7020's DSP48E1 cells natively support 25-by-18-bit multiplication, and the cascade chain accumulates partial products without returning to the general fabric. Each of the 9 partial products takes one cycle. The 768-bit result accumulates in a dedicated register pair. Total latency: 9 cycles. During multiplication, the pipeline stalls — the core issues no other instructions until the multiply completes. Resource cost: approximately 200 lookup tables, 600 flip-flops, and 5 DSP48E1 slices.

The 384-bit barrel shifter implements arbitrary shifts from 0 to 383 positions via 9 stages of 2-to-1 multiplexers (shift amounts of 1, 2, 4, 8, 16, 32, 64, 128, and 256). The general shift completes in one cycle at a cost of approximately 350 lookup tables. The SHR335 instruction — Q335 divmod — does not use the barrel shifter. It is implemented as fixed wiring that routes bits [767:335] of the multiply result register pair to the destination Value register and bits [334:0] to the destination Remainder register. No multiplexer logic is involved. This is the zero-logic divmod.

The 384-bit comparator cascades six 64-bit comparison units. Each unit determines whether its segment is less than, equal to, or greater than the corresponding segment of the other operand. The cascade logic propagates from the most significant segment downward, and the first unequal segment determines the result. All segments are compared in parallel; the cascade selects the result in one cycle. Resource cost: approximately 100 lookup tables and 10 flip-flops.

The cross-multiply control unit implements VDR fraction unification. To test whether fraction a/b equals fraction c/d , it computes a times d and c times b using two invocations of the iterative multiplier, then compares the 768-bit results. If the products are equal, the fractions are equal — this is exact, with no tolerance and no approximation. The unit manages the sequencing of the two multiplications and the final comparison, setting the EQ flag on completion. Total latency: 19 cycles (9 for the first multiply, 9 for the second, 1 for the comparison). Resource cost: approximately 80 lookup tables and 40 flip-flops for the control sequencing.

4.4 Remainder Tree Storage

Each core has a dedicated 2KB block RAM region organized as a flat array of 16 remainder tree nodes. Each node occupies 128 bytes aligned to a power-of-two boundary:

Field	Size	Description
V	384 bits (48 bytes)	Value component
D	384 bits (48 bytes)	Denominator (typically 2^{335} , stored explicitly for generality)

Field	Size	Description
R idx	16 bits (2 bytes)	Index of child node; 0xFFFF indicates no child (closed)
depth	8 bits (1 byte)	Current depth in the remainder tree
flags	8 bits (1 byte)	Status: closed, active, functional marker
padding	28 bytes	Alignment to 128-byte boundary

The maximum depth of 16 matches the scope chain depth limit specified in the system architecture. A free-list maintained by a 4-bit pointer tracks available nodes. The NEST instruction allocates a child node by advancing the free-list pointer, writing initial values, and incrementing the depth counter. The UNNEST instruction deallocates by resetting the node and returning it to the free-list. The PROJECT instruction walks the tree recursively to compute the scalar projection Π — the full rational value collapsed from the tree structure. Each level of the walk takes approximately 3 cycles, so scalar projection of a depth-d tree takes approximately 3d cycles.

In practice, Q335 arithmetic produces shallow remainder trees. Each multiplication nests one level. The Q335 frame provides 2^{335} of headroom, and typical training denominators reach approximately 2^{45} — leaving 290 orders of magnitude of headroom before values exceed the frame. The expected active spill rate (values requiring remainder nesting) is below 1% of all values. The 16-node capacity per core is sufficient for all anticipated workloads.

4.5 Pipeline

The four-stage in-order pipeline processes one instruction per cycle for single-cycle operations.

The fetch stage reads the next instruction from the core's instruction BRAM at the address held in the program counter. The decode stage extracts the opcode, register specifiers, and immediate values from the 32-bit instruction word, and reads source register values from the register file. The execute stage performs the operation — a single-cycle ALU operation, a memory access, or the first cycle of a multi-cycle operation such as multiplication. The writeback stage writes the result to the destination register and updates flags.

The pipeline is in-order with no hazard detection hardware. Multi-cycle operations (multiplication at 9 cycles, cross-multiply at 19 cycles) stall the pipeline — no subsequent instructions issue until the operation completes. This eliminates the complexity of bypass networks and scoreboarding. The software (microprograms loaded by the dispatcher) is responsible for scheduling instructions to avoid data hazards. Since microprograms are short (20 to 60 instructions), fixed sequences written for a known pipeline, this is straightforward and introduces no overhead.

At 150 MHz with single-cycle operations, each core sustains 150 million instructions per

second. With 10 cores, the system sustains 1.5 billion instructions per second for embarrassingly parallel workloads.

4.6 Resource Summary

Component	LUTs	FFs	BRAM (18Kb)	DSP48
Register file	400	6,264	0	0
Adder/subtractor	400	50	0	0
Iterative multiplier	200	600	0	5
Barrel shifter	350	0	0	0
Comparator	100	10	0	0
Cross-multiply control	80	40	0	0
Remainder BRAM	30	10	1	0
Instruction BRAM	30	10	2	0
Pipeline control	200	150	0	0
Unification micro-sequencer	200	100	0	0
Batch interface	100	80	0	0
Core total	2,090	7,314	3	5

5. Instruction Set Architecture

The VDR-Q335 ISA comprises 53 instructions organized into 10 categories. Every instruction is 32 bits wide, encoded in one of four formats.

5.1 Instruction Formats

The R format (register-register) uses bits [31:26] for the opcode, [25:22] for the destination register, [21:18] for source register 1, [17:14] for source register 2, and [13:0] for reserved bits or sub-function specifiers. This format is used by arithmetic, comparison, VDR-specific, and transfer instructions.

The I format (register-immediate) uses bits [31:26] for the opcode, [25:22] for the destination register, [21:18] for source register 1, and [17:0] for an 18-bit immediate value. This format is used by shift, load/store, and immediate-value instructions.

The J format (jump) uses bits [31:26] for the opcode and [25:0] for a 26-bit word-addressed target, providing a 256 MB jump range — far more than the instruction BRAM requires, but consistent with standard RISC encoding practice.

The B format (batch) uses bits [31:26] for the opcode, [25:22] for a register specifier, and [21:0] for batch parameters. This format is used by the batch processing instructions.

5.2 Complete Instruction Table

Opcode	Mnemonic	Format	Cycles	Description
0x00	HALT	—	1	Stop core; set DONE flag
0x01	NOP	—	1	No operation
0x02	JMP	J	1	Unconditional jump
0x03	JEQ	J	1	Jump if EQ flag set
0x04	JLT	J	1	Jump if LT flag set
0x05	JGT	J	1	Jump if GT flag set
0x06	JOV	J	1	Jump if OV flag set
0x07	JCLOSED	J	1	Jump if CLOSED flag set (remainder is zero)
0x08	JDONE	J	1	Jump if batch DONE
0x09	JNE	J	1	Jump if EQ flag not set
0x0A	WADD	R	1	$Vd = Vs1 + Vs2$ (384-bit); sets OV
0x0B	WSUB	R	1	$Vd = Vs1 - Vs2$; sets OV, EQ, LT, GT
0x0C	WMUL	R	9	$[Vd, Rd] = Vs1 \times Vs2$ (384 $\times$ 384 $\rightarrow$ 768); pipeline stalls
0x0D	WCMP	R	1	Compare Vs1 versus Vs2; sets EQ, LT, GT
0x0E	WABS	R	1	$Vd = \text{absolute value of } Vs1$
0x0F	WNEG	R	1	$Vd = \text{two's complement negation of } Vs1$
0x10	AND	R	1	$Vd = Vs1 \text{ bitwise AND } Vs2$
0x11	OR	R	1	$Vd = Vs1 \text{ bitwise OR } Vs2$
0x12	XOR	R	1	$Vd = Vs1 \text{ bitwise XOR } Vs2$
0x13	NOT	R	1	$Vd = \text{bitwise complement of } Vs1$
0x14	SHR	I	1	$Vd = Vs1 \text{ shifted right by immediate (0-383) via barrel shifter}$

Opcode	Mnemonic	Format	Cycles	Description
0x15	SHL	I	1	Vd = Vs1 shifted left by immediate
0x16	SHR335	R	1	Vd = bits [767:335] of [Vs1:Rs1], Rd = bits [334:0]; Q335 divmod via fixed wiring
0x17	SHRN	I	1	General divmod by 2^N for arbitrary N
0x18	NEST	R	2	Allocate remainder child node in BRAM; increment depth
0x19	UNNEST	R	2	Deallocate remainder node; decrement depth
0x1A	RDEPTH	R	1	Id = current remainder tree depth
0x1B	RLOAD	R	2	Load node from remainder BRAM: Vd = node.V, Rd = node.R idx
0x1C	RSTORE	R	2	Store to remainder BRAM node: node.V = Vs1, node.R idx = Rs1
0x1D	PROJECT	R	var	Scalar projection Π : recursive walk of remainder tree; cycles = $3 \times \text{depth}$
0x1E	NORMALIZE	R	var	Normalize VDR triple: sign convention, GCD reduction, closed check; sets CLOSED
0x1F	CROSS MUL	R	19	Two WMUL invocations plus 768-bit compare; VDR fraction unification; sets EQ
0x20	UATOM	R	1	Atom unification via interned integer ID comparison

Opcode	Mnemonic	Format	Cycles	Description
0x21	LDV	I	2	Load 384-bit value from BRAM to Vd
0x22	LDR	I	2	Load 384-bit remainder from BRAM to Rd
0x23	STV	I	2	Store Vs1 to BRAM as 384-bit value
0x24	STR	I	2	Store Rs1 to BRAM as 384-bit remainder
0x25	LDVR	I	2	Load V and R pair together from BRAM
0x26	STVR	I	2	Store V and R pair together to BRAM
0x27	LDI	I	1	Load 18-bit immediate to index register
0x28	SETMA	I	1	Set memory address register from index register plus offset
0x29	BLOAD	B	3	Read 7-byte batch header from BRAM
0x2A	BNEXT	B	1	Increment batch index; set DONE if batch complete
0x2B	BADDR	R	1	Compute DDR address of current batch element
0x2C	BMATCH	R	1	Predicate ID match against batch element; set EQ
0x2D	LD_SHARED	I	2	Read from shared BRAM (Q335 constants, predicate lookup)
0x2E	TSEND	R	1	Write to inter-core transfer register
0x2F	TRECV	R	1	Read from inter-core transfer register
0x30	TWAIT	—	var	Stall until inter-core transfer complete

Opcode	Mnemonic	Format	Cycles	Description
0x31	TDONE	—	1	Signal transfer complete to partner core
0x32	REDUCE ADD	R	5	Send value to reduction network; receive global sum across all cores
0x33	MOVI	I	1	Move 18-bit immediate to Vd bits [17:0]
0x34	MOVHI	I	1	Move 18-bit immediate to Vd bits [35:18]

5.3 Key Instructions

Three instructions deserve particular attention because they implement operations unique to VDR arithmetic and would not appear in a conventional processor ISA.

SHR335 is the Q335 divmod instruction. After a WMUL produces a 768-bit result across a Value-Remainder register pair, SHR335 extracts the quotient (bits above position 335) into the destination Value register and the remainder (bits below position 335) into the destination Remainder register. The implementation is fixed wiring on the FPGA fabric — the bit positions are constants determined at synthesis time, so the “extraction” is the physical routing of wires from source flip-flops to destination flip-flops. No lookup tables, no multiplexers, no switching activity. The one-cycle latency is the pipeline staging delay, not computation time. This instruction makes every Q335 multiplication into a 10-cycle sequence (9 for WMUL, 1 for SHR335) with the second cycle being computationally free.

CROSS_MUL implements VDR fraction unification for Prolog. Given two VDR fractions a/b and c/d , it tests equality by computing $a \times d$ and $c \times b$, then comparing the 768-bit products. This is mathematically exact — cross-multiplication is the standard test for rational equality — and avoids the need to normalize both fractions to a common denominator. The instruction sequences two WMUL operations internally and performs a wide comparison, taking 19 cycles total. This is the hardware cost of the most expensive Prolog unification case (VDR fraction arguments). Atom unification (UATOM, 1 cycle) and integer comparison (WCMP, 1 cycle) handle the common cases at minimal cost.

REDUCE_ADD sends a 384-bit value from the executing core to the binary reduction network and receives the sum of all cores’ contributions. The reduction network is a tree of 384-bit adders — 5 levels for 10 cores, completing in 5 cycles. This instruction enables parallel map-reduce patterns: each core computes a partial result (partial dot product, partial softmax denominator sum, partial aggregation), and REDUCE_ADD produces the global result in 5 additional cycles regardless of the number of cores. The instruction

blocks until the global sum is available.

6. System Architecture

6.1 Core Count and Utilization

Ten VDR-Q335 cores fit on the Xilinx Zynq-7020 (XC7Z020) with the following utilization:

Component	LUTs	FFs	BRAM (18Kb)	DSP48
10 × VDR-Q335 core	20,900	73,140	30	50
Batch dispatcher	2,500	1,800	0	0
AXI register bank	600	500	0	0
AXI DMA engine	2,000	1,200	0	0
Shared BRAM controller	150	80	3	0
Interconnect and infrastructure	1,500	800	2	0
Reduction network	1,200	600	0	0
System total	28,850	78,120	35	50
Available	53,200	106,400	140	220
(Zynq-7020)				
Utilization	54.2%	73.4%	25.0%	22.7%

Flip-flop utilization is the limiting factor at 73.4%, driven by the 384-bit register files (6,264 flip-flops per core). Lookup table utilization at 54.2% leaves 45.8% headroom for timing closure margins, debug instrumentation, and future additions. Block RAM usage is modest at 25% — 30 tiles for core remainder and instruction BRAMs plus 5 for shared and infrastructure. DSP48E1 usage at 22.7% reflects the conservative multiplier design using 5 slices per core.

6.2 Batch Dispatcher

The batch dispatcher is a finite state machine that partitions work across the 10 cores, manages DMA transfers, and collects results. It operates through 11 states:

The idle state (S_IDLE) waits for the host to assert the GO bit in the control register. On assertion, the FSM transitions to the calculation state (S_CALC), which computes the work partition — dividing the input item count across 10 cores with the last core handling any remainder. The program load state (S_LOAD_PROG) initiates a DMA transfer of the selected microprogram from DDR3 to each core's instruction BRAM. The data load state (S_LOAD_DATA) initiates DMA transfers of the input data partitions to each core's working registers or BRAM. The load wait state (S_LOAD_WAIT) polls

the DMA engine’s completion flag. The start state (S_START) asserts the start signal to all cores simultaneously. The wait state (S_WAIT) polls the DONE flags of all 10 cores, proceeding when all are set. The store state (S_STORE) initiates DMA transfers of results from cores to DDR3. The store wait state (S_STORE_WAIT) polls for DMA completion. The next state (S_NEXT) checks whether more data chunks remain — if so, it returns to S_CALC; otherwise it proceeds to the done state (S_DONE), which sets the DONE flag in the status register and returns to S_IDLE.

The dispatcher handles chunking automatically. If the input data exceeds what can be loaded into core BRAMs in a single pass, the dispatcher processes multiple chunks sequentially, accumulating results across passes. The host does not need to manage chunking — it provides the total item count and the DDR3 base addresses, and the dispatcher handles the iteration.

6.3 Shared BRAM

Three 18Kb block RAM tiles (totaling approximately 5.4KB usable) are configured as read-only storage accessible to all cores simultaneously through a shared BRAM controller. The contents are loaded once at system initialization via a DMA transfer triggered by the host writing to the SHARED_GO register.

Offset	Size	Contents
0x0000	1,056 bytes	Q335 constant table: 22 transcendental constants $\times$ 48 bytes each
0x0420	512 bytes	Predicate ID lookup: 256 entries $\times$ 2 bytes
0x0620	2,048 bytes	Grammar enumeration tables: 128 enumerations $\times$ 16 entries $\times$ 1 byte
0x0E20	512 bytes	Configuration: budget thresholds, confidence defaults, operational parameters
0x1020	1,016 bytes	Reserved for future use

The Q335 constant table holds the 22 transcendental constants (π , e , $\ln 2$, $\sqrt{2}$, φ , $\zeta(3)$, and 16 others) as 384-bit numerators. Any core can load a constant via the LD_SHARED instruction in 2 cycles. This supports the QED coefficient computation, FFT twiddle factor generation, and any other operation requiring transcendental constants — without each core needing its own copy.

The predicate ID lookup table maps the 256 most common predicate identifiers (as 16-bit interned integers) to metadata used by the BMATCH instruction for fast predicate matching. This accelerates the common case of Prolog fact queries where the predicate is known and the search is over argument values.

6.4 Reduction Network

The reduction network is a binary tree of 384-bit adders that combines partial results from all 10 cores into a single global result. The tree has 5 levels:

Level 1: 5 adders combine 10 core outputs into 5 partial sums. Level 2: 3 adders combine 5 into 3 (one value passes through). Level 3: 2 adders combine 3 into 2. Level 4: 1 adder combines 2 into 1. Level 5: output register.

Each level takes 1 cycle (using the same carry-select adder design as the core ALU). Total reduction latency: 5 cycles. This is the implementation of the REDUCE_ADD instruction — each core sends its local value to the network, and 5 cycles later every core can read the global sum.

The reduction network supports softmax surrogate computation (sum of squared shifted inputs), vector norm computation (sum of element squares), and any other parallel aggregation operation. Resource cost: approximately 1,200 lookup tables and 600 flip-flops for the entire network.

6.5 Communication

Two AXI bus interfaces connect the FPGA programmable logic to the ARM processing system and shared DDR3 memory.

AXI GP0 is a 32-bit general-purpose interface used for register-mapped control. The host reads and writes 32-bit control registers at memory-mapped addresses. This interface handles operation configuration (writing source addresses, counts, strides, and parameters), status polling (reading busy, done, and error flags), and fast-path Prolog queries (writing predicate and arguments, reading match results). The bandwidth is modest — 32 bits per transfer at the AXI clock rate — but sufficient for control operations that occur once per batch, not once per element.

AXI HP0 is a 64-bit high-performance interface used for DMA bulk transfers between DDR3 and the FPGA. The DMA engine manages burst transfers of input data to core BRAMs and registers, and output results from cores back to DDR3. Double-buffering allows one chunk to be processed while the next is being transferred, hiding DMA latency behind computation for multi-chunk workloads. The 64-bit width at the AXI clock rate provides sufficient bandwidth for the Q335 data widths: each 384-bit value requires 6 transfers, and a burst of 100 values (a typical batch partition) transfers in approximately 600 AXI cycles.

6.6 Memory Map

The Zynq-7020 SoC shares 512MB of DDR3 between the ARM processing system and the FPGA. The memory is partitioned as follows:

Region	Address Range	Size	Contents
Zig kernel	0x0000 0000 – 0x00FF FFFF	16 MB	Runtime, stack, heap
KB fact store	0x0100 0000 – 0x08FF FFFF	128 MB	Knowledge base facts, partitioned for parallel access
Model parameters	0x0900 0000 – 0x10FF FFFF	128 MB	Q335 format parameter storage
Gradient storage	0x1100 0000 – 0x18FF FFFF	128 MB	Gradient vectors for training
Attention matrices	0x1900 0000 – 0x1CFF FFFF	64 MB	QK^T scores and working buffers
DMA buffers	0x1D00 0000 – 0x1FFF FFFF	48 MB	Double-buffered transfer regions
Control registers	0x4000 0000 – 0x4000 00FF	256 B	64 control and status registers
Fast-path registers	0x4000 0100 – 0x4000 01FF	256 B	64 fast-path query registers

The KB fact store is partitioned into 10 equal regions, one per core, enabling parallel fact scanning without memory access contention. The partitioning is performed by the host and communicated to the dispatcher via the source address and stride registers. The host ensures that predicate-indexed facts within a knowledge base are distributed across partitions for balanced load.

7. Register Map

7.1 Control Registers

Offset	Name	Width	Access	Description
0x00	CONTROL	32	Write	Bit 0: GO (auto-clears); Bit 1: abort; Bits [4:2]: operation type (0-5)
0x04	STATUS	32	Read	Bit 0: busy; Bit 1: done; Bit 2: error; Bit 3: DMA active; Bits [13:4]: per-core done flags
0x08	SRC ADDR	32	R/W	DDR3 source base address for input data
0x0C	SRC COUNT	32	R/W	Number of input work items

Offset	Name	Width	Access	Description
0x10	SRC STRIDE	32	R/W	Bytes per input item (48 for single Q335, 96 for pair)
0x14	DST ADDR	32	R/W	DDR3 destination base address for results
0x18	DST STRIDE	32	R/W	Bytes per output item
0x1C	PARAM 0	32	R/W	Operation-specific parameter 0
0x20	PARAM 1	32	R/W	Operation-specific parameter 1
0x24	PARAM 2	32	R/W	Operation-specific parameter 2
0x28	PARAM 3	32	R/W	Operation-specific parameter 3
0x2C	PROG SEL	32	R/W	Microprogram index (0-5); 0xFF for custom program
0x30	PROG ADDR	32	R/W	DDR3 address of custom microprogram (when PROG SEL = 0xFF)
0x34	SHARED SRC	32	R/W	DDR3 source address for shared BRAM load
0x38	SHARED SIZE	32	R/W	Bytes to load into shared BRAM
0x3C	SHARED GO	32	Write	Trigger shared BRAM load from DDR3
0x40	CYCLE COUNT	32	Read	Clock cycles consumed by last operation
0x44	MATCH COUNT	32	Read	Number of matches found or items processed
0x48	ERROR CODE	32	Read	Bits [7:0]: error type; Bits [15:8]: core number; Bits [31:16]: program counter
0x4C	CORE COUNT	32	Read	Hardwired to 10
0x50	VERSION	32	Read	Major.minor.patch encoded as 8.8.16 bits
0x54–0x7C	RESULT 0–9	32 ea.	Read	384-bit scalar result in 12 × 32-bit registers (low to high)

7.2 Fast-Path Query Registers

Offset	Name	Width	Access	Description
0x100	FP PRED- ICATE	32	R/W	Predicate ID to match (16-bit value)
0x104	FP ARG0 LO	32	R/W	First argument, low 32 bits
0x108	FP ARG0 HI	32	R/W	First argument, bits [63:32]
0x10C	FP ARG COUNT	32	R/W	Number of arguments to match (0 = predicate only)
0x110	FP GO	32	Write	Trigger fast-path query
0x114	FP STATUS	32	Read	Bit 0: busy; Bit 1: done; Bits [15:2]: match count
0x118	FP MATCH 0	32	Read	Index of first matching fact
0x11C	FP MATCH 1	32	Read	Index of second matching fact
0x120	FP MATCH 2	32	Read	Index of third matching fact
0x124	FP MATCH 3	32	Read	Index of fourth matching fact

The fast-path interface enables single Prolog queries to execute without DMA overhead. The host writes the predicate ID and optional arguments to registers, asserts FP_GO, and polls FP_STATUS. The dispatcher assigns the query to available cores (partitioning the fact table across them), cores execute the BMATCH instruction sequence, and matching fact indices are written to the result registers. For a knowledge base with 200 facts distributed across 10 cores, each core scans 20 facts. At approximately 8 cycles per non-matching fact and 12 cycles per match, the worst-case scan takes approximately 240 cycles — 1.6 microseconds at 150 MHz. Typical queries with selective predicates complete faster due to the BMATCH instruction's early termination.

8. Microprograms

Six microprograms are pre-loaded into the system and selectable via the PROG_SEL register. Each is a short, fixed instruction sequence loaded into core instruction BRAMs

by the dispatcher before execution begins.

8.1 Fact Match (`prog_fact_match`)

Purpose: scan a partition of the fact table for facts matching a specified predicate ID, optionally with argument matching.

The program loads the target predicate ID from the batch control registers, then iterates through the assigned fact partition. For each fact, the `BMATCH` instruction compares the fact's predicate ID (a 16-bit interned integer) against the target. On match, the fact index is written to the result buffer. On non-match, the program advances to the next fact via `BNEXT`. Argument matching, when enabled, uses `WCMP` for integer arguments, `UATOM` for atom arguments, and `CROSS_MUL` for VDR fraction arguments.

Estimated size: approximately 40 instructions. Per-fact latency: 8 cycles (no match) to 12 cycles (match, with index write). For 200 facts across 10 cores: approximately 160 cycles per core, approximately 1.1 microseconds total.

8.2 Q335 Add and SGD Update (`prog_q335_add`)

Purpose: batch addition or subtraction of Q335 value pairs, used for SGD parameter updates ($W \leftarrow W - lr \times grad$).

The program loads the learning rate from a parameter register (set by the host), then iterates through the assigned slice of the parameter vector. For each element, it loads the gradient, performs `WMUL` (learning rate $\times$ gradient, 9 cycles), performs `SHR335` (Q335 divmod, 1 cycle, zero logic), and performs `WSUB` (parameter $-$ update, 1 cycle). The result is stored back to the parameter's DDR location. If the `SHR335` produces a nonzero remainder, the `NEST` instruction allocates a remainder node, but this is expected to occur in less than 1% of updates.

Estimated size: approximately 20 instructions. Per-element latency: approximately 15 cycles (dominated by the 9-cycle `WMUL`). For 100,000 parameters across 10 cores: 10,000 elements per core $\times$ 15 cycles = 150,000 cycles per core = 1 millisecond at 150 MHz.

8.3 Q335 Multiply with Remainder (`prog_q335_mul`)

Purpose: batch Q335 multiplication with full remainder handling, used for attention score computation and general exact arithmetic.

The program performs `WMUL` followed by `SHR335` for each pair of input values. When the remainder is nonzero (tested by the `JCLOSED` conditional jump), the program executes `NEST` to allocate a remainder tree node, `RSTORE` to save the remainder value, and updates the depth counter. The program handles nesting up to the core's 16-node remainder BRAM capacity.

Estimated size: approximately 50 instructions. Per-element latency: approximately 12 cycles minimum (`WMUL` + `SHR335` + store), more with remainder nesting. The remainder path adds approximately 6 cycles per nesting level.

8.4 Dot Product (`prog_dot_product`)

Purpose: compute the exact Q335 dot product of two vectors, used for attention score computation (one element of QK^T).

The program iterates through paired elements of the two input vectors. For each pair, it performs WMUL (9 cycles) and SHR335 (1 cycle) to compute the Q335 product, then WADD (1 cycle) to accumulate into a running sum. After processing all elements, the accumulator holds the exact dot product. For multi-core reduction (when the dot product is partitioned across cores), REDUCE_ADD (5 cycles) produces the global sum.

Estimated size: approximately 35 instructions. Per-dimension latency: approximately 14 cycles (WMUL + SHR335 + WADD + loop control). For hidden dimension $H = 64$: approximately 896 cycles per dot product = 5.97 microseconds at 150 MHz. For an attention matrix of sequence length S : S dot products, distributable across cores by assigning different query positions to different cores.

8.5 Constraint Evaluation (`prog_constraint_eval`)

Purpose: evaluate a single Prolog constraint against the current knowledge base state.

The program loads the constraint's condition (a Prolog goal) from the batch parameters. It uses BMATCH to find relevant facts, WCMP and CROSS_MUL for value comparisons, and conditional jumps to evaluate logical structure. The result (satisfied or violated) is written to the result buffer with the constraint identifier.

Estimated size: approximately 60 instructions. Per-constraint latency: variable, depending on the number of facts consulted and the complexity of the condition. Typical constraints (sum-to-one verification, budget threshold check) require 20-40 cycles. Complex constraints with multiple Prolog goals may require 100 or more cycles.

Since constraints are independent, the dispatcher assigns one constraint per core. With 10 cores and 15 constraints (a typical lifecycle phase), the batch completes in two passes — 10 constraints in the first pass, 5 in the second. Total time is determined by the slowest constraint in each pass, not the sum.

8.6 Softmax Surrogate (`prog_softmax_surr`)

Purpose: compute the rational surrogate softmax over a vector of logits, where each output = $(z - \max + c)^2 / \sum (z - \max + c)^2$.

The program operates in three phases. In the map phase, each core processes its assigned slice of the logit vector: for each logit, it subtracts the maximum (pre-computed by the host or by a prior REDUCE pass), adds the shift parameter c , and computes the square via WMUL + SHR335. The partial sum of squares is accumulated locally. In the reduce phase, REDUCE_ADD produces the global denominator (sum of all squares) across all cores in 5 cycles. In the normalize phase, each core divides its local squared values by the global denominator to produce the final softmax outputs.

Estimated size: approximately 30 instructions per phase. Total latency for 100 logits across 10 cores: approximately 500 cycles = 3.3 microseconds at 150 MHz. The result sums to exactly one — not approximately one, exactly one — because the numerators are the individual squares and the denominator is their exact sum.

9. Host-FPGA Integration

9.1 Division of Responsibilities

The Zynq-7020 SoC places an ARM Cortex-A9 dual-core processor and the FPGA fabric on the same die, sharing DDR3 memory. The Zig VDR-LLM-Prolog runtime runs on the ARM cores and handles all control-plane operations.

The host is responsible for the orchestrated inference loop (assess, formalize, execute, store), knowledge base tree management (creation, deletion, scoping, mounting, connecting), session management (snapshots, clones, disposal), grammar parsing and generation, command token parsing and dispatch, language model forward and backward pass orchestration, remainder tree operations beyond depth 4 (rare, complex recursive traversal), functional remainder resolution (Newton iteration, Taylor series, and other sequential convergent computations), and all control flow decisions.

The FPGA is responsible for parallel Prolog fact matching across knowledge base partitions, Q335 batch arithmetic (addition, subtraction, multiplication, divmod), row-parallel attention score computation (QK^T dot products), parallel constraint evaluation, parallel denominator budget checking, softmax surrogate computation (parallel map-reduce), and VDR fraction cross-multiply comparison for Prolog unification.

9.2 Communication Protocol

A typical accelerated operation follows this sequence:

The host writes the operation type to the CONTROL register (bits [4:2] select among the 6 pre-loaded microprograms or custom program mode). The host writes the DDR3 source address, item count, stride, destination address, and any operation-specific parameters to the configuration registers. The host sets the GO bit in the CONTROL register.

The batch dispatcher reads the configuration, computes the work partition across 10 cores, and initiates DMA transfers to load the selected microprogram into each core's instruction BRAM and the input data into core registers or BRAM. Once all transfers complete, the dispatcher asserts the start signal to all cores simultaneously. The cores execute their microprograms in parallel. As each core completes, it sets its DONE flag. The dispatcher polls until all 10 DONE flags are set.

The dispatcher initiates DMA transfers of results from core output regions to the DDR3 destination address. Once transfers complete, the dispatcher sets the DONE flag in the STATUS register and returns to idle.

The host polls the STATUS register (or receives an interrupt, if configured) to detect completion, then reads results from DDR3 or from the RESULT registers for scalar outputs.

For latency-sensitive operations — particularly single Prolog queries during the inference loop — the fast-path register interface bypasses the batch dispatcher entirely. The host writes the predicate ID and arguments to the fast-path registers and asserts FP_GO. The FPGA dispatches the query directly to available cores, which scan their fact partitions and write matching indices to the fast-path result registers. The host reads FP_STATUS for the match count and FP_MATCH registers for the matching fact indices. Round-trip latency for a 200-fact knowledge base: approximately 200 nanoseconds for the FPGA scan plus AXI register access overhead — under 500 nanoseconds total.

9.3 Bit-Identical Verification

Every operation performed by the FPGA must produce results identical to the software implementation. This is not a goal — it is a constraint enforced by the system architecture. The VDR-LLM-Prolog specification defines every operation through IOSE declarations (Inputs, Outputs, Side Effects) with declared mathematical properties. The FPGA implements a subset of these operations in parallel hardware. The results must match the software path exactly — same Value, same Denominator, same Remainder at every level of the tree.

Verification follows a two-stage process. During development, every microprogram is tested by running the same inputs through both the FPGA accelerator and the Python reference implementation, comparing results at the bit level. During deployment, the host can selectively route operations to either the software path or the FPGA path without affecting correctness — both produce identical results. The existing 884-test validation suite serves as the acceptance test: every test that passes against the software implementation must also pass against the FPGA-accelerated implementation.

This constraint is achievable because Q335 arithmetic is deterministic integer computation. There are no floating-point rounding modes, no compiler optimization reordering, no hardware-dependent transcendental approximations. The same integer inputs processed by the same integer operations produce the same integer outputs regardless of whether those operations execute in Python on an x86 processor, in Zig on an ARM processor, or in a custom pipeline on FPGA fabric.

10. What Does Not Belong on FPGA

The boundary between host and FPGA responsibilities is determined by the nature of each operation, not by a desire to maximize hardware utilization. Several categories of VDR-LLM-Prolog operations are better served by software.

Deep remainder tree operations beyond approximately depth 4 involve complex recursive traversals with data-dependent branching. The FPGA's remainder BRAM supports up to 16 levels, but deep traversals are inherently sequential (each parent depends on its

children) and rare (Q335 values almost never exceed the frame). Software handles these infrequent cases without performance impact.

Functional remainder resolution — evaluating Newton iteration, Taylor series, Borwein acceleration, and other convergent computations — is inherently sequential. Each iteration depends on the result of the previous iteration. The FPGA can accelerate individual arithmetic operations within each iteration, but the iteration control, convergence checking, and depth management are control-plane decisions best handled in software.

Knowledge base tree structure management — creating and deleting knowledge bases, managing the scope chain, handling mounts and connections, resolving dotted paths to integer identifiers — involves hash table operations, pointer management, and complex state transitions. These are irregular, pointer-chasing operations with poor parallelism. The Zig runtime handles them efficiently with hash maps and arena allocators.

Grammar matching and scoring involves Prolog pattern matching (potentially FPGA-acceleratable for the deterministic components) combined with language model assessment for the “best-when” relevance component. The mixed deterministic-probabilistic nature makes this unsuitable for pure hardware acceleration unless profiling reveals the deterministic matching to be a bottleneck.

Session snapshot and restore operations involve bulk memory copies managed by arena allocators. The Zig runtime handles these efficiently, and the operations are infrequent (snapshots occur at meaningful state transitions, not every turn).

The language model forward and backward passes remain on the host processor. While the FPGA accelerates individual operations within these passes (attention scores, softmax, parameter updates), the overall pass orchestration — layer sequencing, residual connections, loss computation, gradient accumulation — requires control flow that the host manages.

11. Scaling

The VDR-Q335 core design is parameterized by the number of cores instantiated. The same core, the same dispatcher, the same ISA, and the same microprograms scale to larger FPGA devices by increasing the core count.

Platform	Available LUTs	Available DSP48	Available BRAM	Max Cores	Limiting Resource
Zynq-7020	53,200	220	140	10	FF (73.4%)
Zynq-7045	218,600	900	545	60	LUT (71%)
Zynq-7100	444,000	2,020	1,090	120	DSP (30%)
UltraScale+ ZU7EV	504,000	1,728	1,080	130	DSP (38%)

Platform	Available LUTs	Available DSP48	Available BRAM	Max Cores	Limiting Resource
UltraScale+ ZU19EG	1,143,000	1,968	1,968	200+	Balanced

Scaling is linear for embarrassingly parallel operations: 60 cores scan facts 6 times faster than 10 cores, compute 6 times more parameter updates per cycle, and process 6 times more attention elements simultaneously. The reduction network grows logarithmically — 60 cores require 6 levels instead of 5, adding one cycle to the REDUCE_ADD latency.

The Zynq-7020 at approximately \$200 serves as the development and validation platform. It proves the architecture, validates bit-identical results against the software reference, and provides acceleration for small-scale experiments. Production deployments targeting larger knowledge bases, longer training runs, or lower-latency inference would use larger devices where the same architecture delivers proportionally higher throughput.

The core design also targets non-Zynq FPGA platforms. Any device with DSP48-equivalent multiply-accumulate blocks, block RAM, and an AXI or similar bus interface can host VDR-Q335 cores. The ISA and microprograms are independent of the FPGA vendor — only the physical implementation (lookup table mapping, routing, timing constraints) changes.

12. Development Path

Phase 1: Prolog Accelerator (Weeks 1–8)

Build the minimum useful accelerator: parallel Prolog fact matching across knowledge base partitions.

Port the core pipeline to 384-bit working width. Implement the WADD, WSUB, WCMP, WABS, and WNEG instructions — single-cycle operations using the carry-select adder and comparator. Implement the BMATCH instruction for predicate ID matching. Implement the batch dispatcher FSM with DMA load and store. Implement the AXI register bank with control and fast-path registers. Write the Zig host interface: register writes for configuration, polling for completion, result reading.

Test with VDR-LLM-Prolog Stage 1 knowledge bases (5 KBs, 50 facts) and Stage 2 (15 KBs, 200 facts). Verify that fact query results from the FPGA match the software implementation exactly for all test cases.

Deliverable: Prolog queries on FPGA returning matching fact indices to the Zig host, with measured speedup versus software scan.

Phase 2: Q335 Arithmetic (Weeks 9–14)

Add wide multiplication and Q335-specific operations.

Implement the 128-bit iterative multiplier using DSP48E1 cascade. Implement WMUL (9-cycle 384×384 multiply), SHR335 (zero-logic Q335 divmod), SHRN (general power-of-two divmod), and the barrel shifter. Implement NEST, UNNEST, RLOAD, RSTORE, RDEPTH for remainder tree management. Write the `prog_q335_add` and `prog_q335_mul` microprograms. Test batch Q335 parameter updates against the software reference.

Test with VDR-4 training operations: a single exact SGD step with learning rate as a Q335 fraction, verifying that every parameter update matches the Python reference.

Deliverable: batch Q335 arithmetic on FPGA with remainder handling, validated against the software path.

Phase 3: Attention Acceleration (Weeks 15–20)

Add the operations needed for exact transformer inference.

Implement the reduction network (binary tree of 384-bit adders, 5 levels). Implement REDUCE_ADD for global summation across cores. Write the `prog_dot_product` microprogram for row-parallel attention score computation. Write the `prog_softmax_surr` microprogram for parallel surrogate softmax with map-reduce. Test with VDR-4 Tiny-TransformerLM: compute exact attention scores on FPGA, verify that attention weights sum to exactly one, verify that all values match the software reference.

Deliverable: exact attention scores and surrogate softmax computed on FPGA, integrated with the host-side transformer inference loop.

Phase 4: Unification Engine (Weeks 21–26)

Add hardware support for complete Prolog term unification.

Implement CROSS_MUL (19-cycle VDR fraction unification via cross-multiplication). Implement UATOM (1-cycle atom unification via interned integer comparison). Implement the unification micro-sequencer for multi-goal rule evaluation with variable binding management in BRAM. Write the `prog_constraint_eval` microprogram for parallel constraint checking. Test with VDR-9 inference notebook Prolog queries: rule evaluation, backtracking, constraint verification.

Deliverable: complete Prolog query evaluation on FPGA for common term types (atoms, integers, VDR fractions, knowledge base references), with variable binding and multi-goal sequencing.

Phase 5: Integration and Profiling (Weeks 27–32)

Full integration with the VDR-LLM-Prolog Zig runtime on the ARM host.

Profile end-to-end operation: measure time spent in host versus FPGA for each operation type across the orchestrated inference loop. Optimize DMA transfer patterns — implement double-buffering for overlapping computation and transfer. Test with the VDR-11 Stage 4 full lifecycle pipeline: forward pass, loss computation, backward pass, parameter

update, checkpoint save, constraint verification. Measure total speedup versus the pure-software Zig implementation for representative workloads: SRE investigation, training run, knowledge base ingestion.

Deliverable: benchmarked VDR-LLM-Prolog system with FPGA acceleration, profiling data identifying remaining bottlenecks, and performance comparison against software-only execution.

13. Verification Strategy

Correctness verification for the FPGA implementation operates at three levels.

At the instruction level, every instruction in the ISA is tested individually with boundary-case inputs: zero operands, maximum-value operands, operands producing overflow, operands producing zero and nonzero remainders. The result of each instruction execution on the FPGA is compared bit-by-bit against the result computed by the Python reference implementation. Any divergence is a bug in the hardware implementation.

At the microprogram level, each of the six microprograms is tested with representative workloads. The `prog_fact_match` microprogram is tested against knowledge bases of varying sizes (10, 100, 1000, 10000 facts) with predicates that match zero, some, and all facts. The `prog_q335_add` microprogram is tested with parameter vectors including values near zero, near the Q335 maximum, and values that produce remainder nesting. The `prog_dot_product` and `prog_softmax_surr` microprograms are tested against the VDR-4 attention test cases, verifying that attention weights sum to exactly one. In every case, the FPGA output is compared against the software reference.

At the system level, the project's complete 884-test validation suite is executed against the FPGA-accelerated system. Tests that exercise operations routed to the FPGA must produce identical results to the software-only execution. The test suite spans 37 domains — number theory, polynomial algebra, matrix decomposition, signal processing, quantum mechanics, orbital mechanics, and 31 others — providing broad coverage of arithmetic edge cases that might reveal hardware implementation errors.

Additionally, the system supports a shadow mode where both the software path and the FPGA path execute the same operation and their results are compared automatically. This mode is used during initial deployment and can be enabled selectively for operations where correctness is critical (financial calculations, safety-relevant constraint checks). Shadow mode doubles the computation cost but provides real-time verification that the hardware path remains correct.

14. Limitations and Honest Boundaries

The FPGA acceleration does not make VDR-LLM-Prolog practical for production-scale language model training. A 100-million parameter model requires approximately 4.8

gigabytes of storage at 384 bits per parameter — exceeding the DDR3 capacity of the Zynq-7020 (512 MB total, with only 128 MB allocated to parameter storage). The FPGA accelerates the arithmetic operations within training, but the memory constraint limits the model size to thousands or tens of thousands of parameters. This is sufficient for the toy-scale validation that VDR-4 specifies but not for practical language model development. Production-scale training requires the GPU implementation specified in VDR-18.

The 150 MHz clock frequency of the FPGA fabric is approximately 10 to 20 times slower than the clock frequencies of modern GPUs (1.5 to 2.5 GHz). The FPGA compensates with architectural advantages — zero-logic divmod, deterministic pipeline, no memory hierarchy complexity — but raw clock speed limits absolute throughput. The FPGA’s advantage is parallel exact integer arithmetic at moderate scale, not competition with GPU tensor cores at datacenter scale.

The 10-core count on the Zynq-7020 provides modest parallelism. Larger FPGAs scale to 60, 120, or 200+ cores, but even 200 cores operating at 150 MHz do not match the thousands of cores operating at GHz frequencies in modern GPUs. The FPGA targets a different deployment profile: embedded systems, edge devices, development platforms, and environments where GPU hardware is unavailable or where power consumption must be minimized.

The ISA does not include instructions for floating-point operations. This is by design — the VDR system has no floats anywhere — but it means the FPGA accelerator cannot be repurposed for conventional neural network inference. It is a single-purpose accelerator for exact integer arithmetic.

Functional remainder resolution (Newton iteration, Taylor series, and other convergent computations) remains in software. The FPGA accelerates the per-step arithmetic but cannot manage the iterative control flow efficiently in hardware. This means operations like computing $\sqrt{2}$ to 100 digits still execute on the host processor, with the FPGA providing no acceleration for the sequential iteration structure. If profiling reveals functional remainder resolution as a bottleneck, a future revision could add a dedicated iteration controller, but the current design prioritizes the embarrassingly parallel operations that dominate total computation time.

The hardware design has been resource-estimated and architecturally specified but has not been synthesized, placed, or routed on an actual FPGA. The utilization percentages and timing estimates are based on component-level resource counting against the Zynq-7020’s published specifications. Actual utilization after synthesis may differ due to routing congestion, timing closure requirements, and tool-specific optimizations. The 45.8% LUT headroom and 26.6% FF headroom provide margin for these effects, but synthesis results may require design adjustments — particularly for the 384-bit carry-select adder, which has a long critical path that may limit achievable clock frequency.

15. Prior Work

The hardware architecture patterns in this paper — batch dispatching, AXI register-mapped control, shared read-only BRAM, four-stage in-order pipeline, DMA bulk transfer with double-buffering — are standard system-on-chip engineering practices documented extensively in the Xilinx design methodology literature. The specific application of these patterns to a 384-bit integer arithmetic processor with Q335-specific instructions, remainder tree management, and Prolog unification support is novel.

An earlier hardware specification (CKS-MATH-145/148, unpublished) described an FPGA implementation for a system called VFR (Value-Fragment-Remainder) using a narrower representation ([V:i32, R:i8]) on the same Zynq-7020 target. The mathematical framework underlying that system was subsequently invalidated. The hardware architecture patterns — batch processing FSM, AXI interface design, core pipeline structure, shared BRAM organization — are independent of the mathematical framework and transfer to the VDR design. The specific adaptations required for VDR (384-bit working width, Q335 divmod instruction, remainder tree BRAM, cross-multiply unification unit) are documented in this paper.

Appendix A: Parallelism Classification

A.1 Embarrassingly Parallel (No Dependencies Between Cores)

Operation	Data Unit	Cores Useful	Notes
Q335 addition (bulk)	Pair of numerators	All	One wide add per core per cycle
Constraint checking	Individual constraint	All	Each constraint evaluated independently
Softmax surrogate (map phase)	Individual logit	All	Square and accumulate independently
Denominator budget check	Individual parameter	All	Compare denominator bits against budget
Q335 reprojection	Individual parameter	All	Round and compute error bound
Fact query by predicate	KB partition	All	Each core scans its partition

Operation	Data Unit	Cores Useful	Notes
Bitset operations	Individual bits	All	Set, test, and count in parallel

A.2 Parallel with Reduction

Operation	Parallel Phase	Reduction Phase	Reduction Latency
Softmax normalization	Compute each term	Sum all terms (REDUCE ADD)	5 cycles
Matrix-vector multiply	Each row independently	Collect results	0 (independent)
Dot product	Partial products	Sum partials (REDUCE ADD)	5 cycles
Determinant (Gaussian)	Row operations within step	Pivot selection between steps	1 cycle
List aggregates	Partial aggregates	Combine (REDUCE ADD)	5 cycles

A.3 Sequential (Limited Parallelism)

Operation	Sequential Dependency	Acceleration Strategy
Prolog backtracking	Each binding depends on previous	Pipeline different queries; parallelize across independent goals
Newton iteration	Each step depends on previous	Accelerate per-step arithmetic on FPGA
Remainder tree traversal	Parent-child dependency	Depth-first hardware walker in BRAM
Scope chain resolution	Walk parent pointers	Short chain (max 16); pipeline with prefetch

Appendix B: Instruction Encoding Detail

B.1 R Format (Register-Register)

[31:26] opcode (6 bits)

[25:22] Vd or Rd destination (4 bits, selects V0-V7 or R0-R7)

[21:18] Vs1 or Rs1 source 1 (4 bits)

[17:14] Vs2 or Rs2 source 2 (4 bits)

[13:0] reserved / sub-function (14 bits)

Used by: WADD, WSUB, WMUL, WCMP, WABS, WNEG, AND, OR, XOR, NOT, SHR335, NEST, UNNEST, RDEPTH, RLOAD, RSTORE, PROJECT, NORMALIZE, CROSS_MUL, UATOM, BADDR, BMATCH, TSEND, TREC, REDUCE_ADD

B.2 I Format (Register-Immediate)

[31:26] opcode (6 bits)

[25:22] Vd or Rd destination (4 bits)

[21:18] Vs1 or Rs1 source (4 bits)

[17:0] immediate value (18 bits, signed or unsigned per instruction)

Used by: SHR, SHL, SHRN, LDV, LDR, STV, STR, LDVR, STVR, LDI, SETMA, LDSSHARED, MOVI, MOVHI

B.3 J Format (Jump)

[31:26] opcode (6 bits)

[25:0] target address (26 bits, word-addressed)

Used by: JMP, JEQ, JLT, JGT, JOV, JCLOSED, JDONE, JNE

B.4 B Format (Batch)

[31:26] opcode (6 bits)

[25:22] register specifier (4 bits)

[21:0] batch parameters (22 bits)

Used by: BLOAD, BNEXT

Appendix C: Shared BRAM Contents

C.1 Q335 Constant Table (22 entries $\times$ 48 bytes = 1,056 bytes)

Index	Constant	Symbol	Numerator Bits
0	Pi	π	337
1	Euler's number	e	337
2	Natural log 2	$\ln(2)$	335
3	Square root 2	$\sqrt{2}$	336
4	Golden ratio	φ	336
5	Pi squared	π^2	339
6	Pi cubed	π^3	340
7	Pi to the fourth	π^4	342
8	e to the pi	e^{π}	340
9	$\ln^2(2)$	$\ln(2)^2$	334
10	$\ln^4(2)$	$\ln(2)^4$	333
11	Natural log 3	$\ln(3)$	336
12	Natural log 5	$\ln(5)$	336
13	Natural log 10	$\ln(10)$	337
14	Square root 3	$\sqrt{3}$	336
15	Square root 5	$\sqrt{5}$	337
16	Square root 7	$\sqrt{7}$	337
17	Zeta(2)	$\zeta(2)$	336
18	Zeta(3)	$\zeta(3)$	336
19	Zeta(5)	$\zeta(5)$	336
20	$\text{Li}_4(1/2)$	$\text{Li}_4(1/2)$	335
21	Catalan's constant	G	335

All 22 constants share the implicit denominator 2^{335} . Each is stored as a 384-bit (48-byte) value in the shared BRAM, accessible to any core via the LDSHARED instruction. The full-precision numerator values are specified in [HOWL-MATH-4-2026] and verified against arbitrary-precision reference implementations at 100 decimal digits.

Appendix D: Performance Projections

D.1 Operation Latencies at 150 MHz

Operation	Cycles	Time	Cores Used	Total Throughput
Single Q335 add	1	6.7 ns	1	150 M ops/sec/core
Single Q335 multiply	9	60 ns	1	16.7 M ops/sec/core

Operation	Cycles	Time	Cores Used	Total Throughput
Single Q335 divmod (SHR335)	1	6.7 ns	1	150 M ops/sec/core
Single fraction unification	19	127 ns	1	7.9 M ops/sec/core
Fact query (200 facts)	~160	~1.1 μ s	10	~910 K queries/sec
Dot product (H=64)	~896	~5.97 μ s	1	167 K dot products/sec
Softmax surrogate (100 logits)	~500	~3.3 μ s	10	~300 K softmax/sec
Batch SGD update (100K params)	~150K	~1.0 ms	10	~1,000 updates/sec
Constraint batch (15 constraints)	~200	~1.3 μ s	10	~770 K batches/sec

D.2 Scaling Across Platforms

Platform	Cores	Fact Query (200 facts)	Batch SGD (100K params)	Softmax (100 logits)
Zynq-7020	10	~1.1 μ s	~1.0 ms	~3.3 μ s
Zynq-7045	60	~0.18 μ s	~170 μ s	~0.6 μ s
Zynq-7100	120	~0.09 μ s	~83 μ s	~0.3 μ s
UltraScale+200 ZU19EG		~0.05 μ s	~50 μ s	~0.17 μ s

Appendix E: Remainder Node Layout

Each core's 2KB remainder BRAM holds 16 nodes at 128 bytes each.

Byte Offset	Field	Size	Description
0–47	V	384 bits	Value component
48–95	D	384 bits	Denominator (typically 2 ³³⁵ stored explicitly)

Byte Offset	Field	Size	Description
96–97	R_idx	16 bits	Index of child node (0xFFFF = no child = closed)
98	depth	8 bits	Current depth in remainder tree
99	flags	8 bits	Bit 0: closed; Bit 1: active; Bit 2: functional marker
100–127	padding	28 bytes	Alignment to 128-byte boundary

The free-list is maintained by a 4-bit pointer (values 0–15) stored in a dedicated register. NEST increments the pointer and initializes the new node. UNNEST clears the node and decrements the pointer. The pointer value also serves as the current depth counter, eliminating the need for a separate depth register.

Nodes are accessed by index: the BRAM address of node i is the base address plus $i \times 128$. The R_idx field of a parent node holds the index of its child. Traversal from parent to child is one BRAM read (2 cycles via RLOAD). The maximum chain length of 16 matches the system-wide scope chain depth limit.

References

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Denominator, Remainder. Exact Finite Arithmetic in Irreducible Triple Form. DOI: 10.5281/zenodo.15302702

[[HOWL-MATH-4-2026](#)] Universal Power-of-Two Basis. Q335 constant representation and verification.

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. From Arithmetic Library to Working Transformer in 24 Modules.

[[HOWL-VDR-13-2026](#)] VDR in Physical Computation. Exact Arithmetic Where It Matters.

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. A Complete System Specification for Exact Arithmetic Language Models with Structural Provenance. DOI: 10.5281/zenodo.20232194

[[HOWL-VDR-18-2026](#)] VDR-LLM-Prolog: Performance. Integer Arithmetic on GPU Hardware.

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Github: <https://github.com/ghowland/papers/tree/main/papers/VD-VDR-21-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>

[[HOWL-VDR-13-2026](#)] VDR in Physical Computation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-13-2026>

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026>

[[HOWL-VDR-18-2026](#)] VDR-LLM-Prolog: Performance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-18-2026>